

#SKILL - 6 → Spring MVC Web Request Handling Demo

Prerequisites:

- General Idea on Spring Web MVC and Form Handling
- Basic understanding of Spring Boot
- Understanding of HTTP methods (GET/POST)

An online library system wants to allow users to explore books, view specific book details, submit feedback, and interact with the system through various types of requests. To support these features, the backend team needs to implement multiple **Spring MVC endpoints** demonstrating **request mappings**, **path variables**, **request parameters**, and **JSON data handling**. Your task is to create a **Spring Boot MVC controller** that exposes different demo operations using **@RestController**, **@GetMapping**, **@PostMapping**, and **@PathVariable**.

Tasks:

1. Create a controller class named **LibraryController**.
2. Create a method mapped to **/welcome** that returns a welcome message.
3. Create a method mapped to **/count** that returns an integer representing total books.
4. Create a method mapped to **/price** that returns a sample book price as **double**.
5. Create a method mapped to **/books** using **@GetMapping** that returns a list of book titles.
6. Create a method mapped to **/books/{id}** using **@GetMapping** and return book details using **@PathVariable**.
7. Create a method mapped to **/search** using **@GetMapping** that accepts a **request parameter** (title) and returns a confirmation message.
8. Create a method mapped to **/author/{name}** using **@GetMapping** that returns a formatted message with the **author's name**.
9. Create a method mapped to **/addbook** using **@PostMapping** that accepts a **Book object** from the request body and adds it to an **in-memory list**.
10. Create a method mapped to **/viewbooks** using **@GetMapping** that returns all added **Book objects**.

SPRING MVC WEB REQUEST HANDLING DEMO PREREQUISITES

1)File → New → Spring Starter Project

Field	Value
Name	SpringMVCWeb
Type	Maven
Packaging	Jar
Java Version	17
Group	com.klu

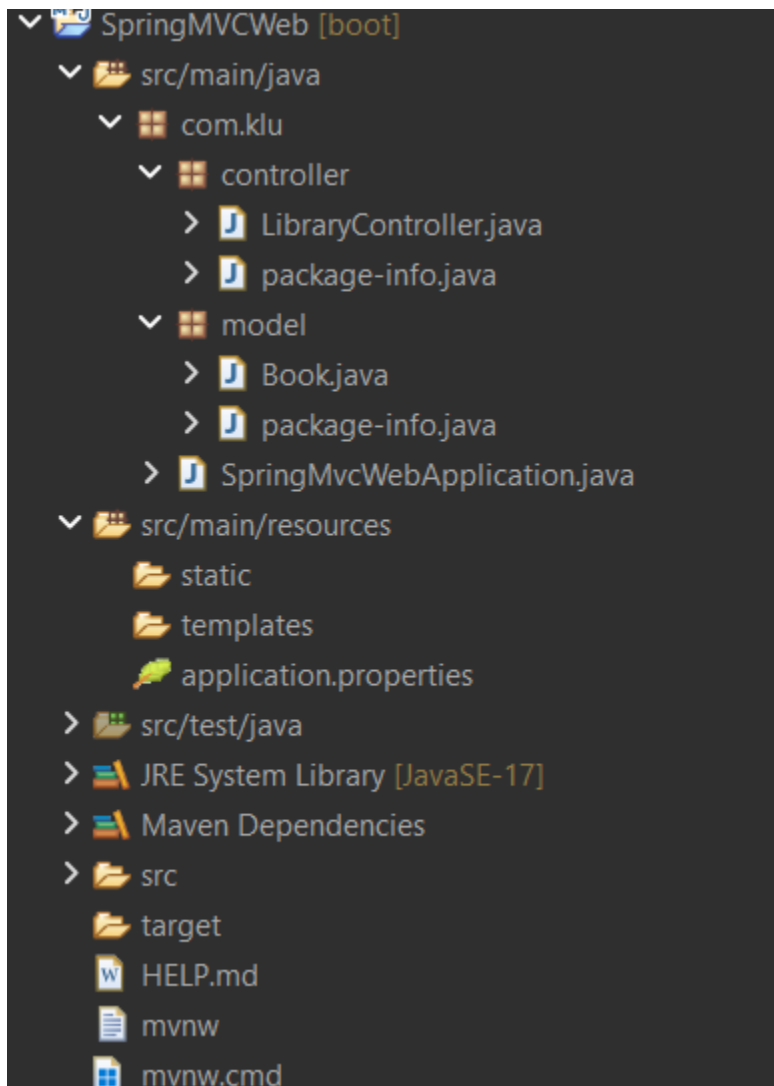
Click **Next**

Add Dependencies

Select: Spring Web

Click **Finish**

2) Project Structure



Model ->Book :

package com.klu.model;

public class Book {

private int id;

private String title;

private String author;

private double price;

public Book() {}

public Book(int id, String title, String author, double price) {

 this.id = id;

 this.title = title;

 this.author = author;

 this.price = price;

```

    }

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public String getTitle() {
        return title;
    }

    public void setTitle(String title) {
        this.title = title;
    }

    public String getAuthor() {
        return author;
    }

    public void setAuthor(String author) {
        this.author = author;
    }

    public double getPrice() {
        return price;
    }

    public void setPrice(double price) {
        this.price = price;
    }
}

```

Controller -> LibraryController:

```
=====
```

```
package com.klu.controller;
```

```

import org.springframework.web.bind.annotation.*;
import java.util.*;
import com.klu.model.Book;

```

```

@RestController
public class LibraryController {

    List<Book> bookList = new ArrayList<>();

    @GetMapping("/welcome")
    public String welcome() {
        return "Welcome to Online Library Systems";
    }

    @GetMapping("/count")
    public int countBooks() {
        return 100;
    }

    @GetMapping("/price")
    public double bookPrice() {
        return 499.99;
    }

    @GetMapping("/books")
    public List<String> getBooks() {
        return Arrays.asList("Java Programming","Spring Boot Guide","Data Structures");
    }

    @GetMapping("/books/{id}")
    public String getBookById(@PathVariable int id) {
        return "Details of Book with ID: " + id;
    }

    @GetMapping("/search")
    public String searchBook(@RequestParam String title) {
        return "Searching for book: " + title;
    }

    @GetMapping("/author/{name}")
    public String getAuthor(@PathVariable String name) {
        return "Books written by Author: " + name;
    }

    @PostMapping("/addbook")
    public String addBook(@RequestBody Book book) {
        bookList.add(book);
    }
}

```

```
        return "Book added successfully!";
    }

    @GetMapping("/viewbooks")
    public List<Book> viewBooks() {
        return bookList;
    }
}
```

=====

Run the Application:

Run As → Spring Boot App

Open Postman app:

Click **New** → **HTTP Request**

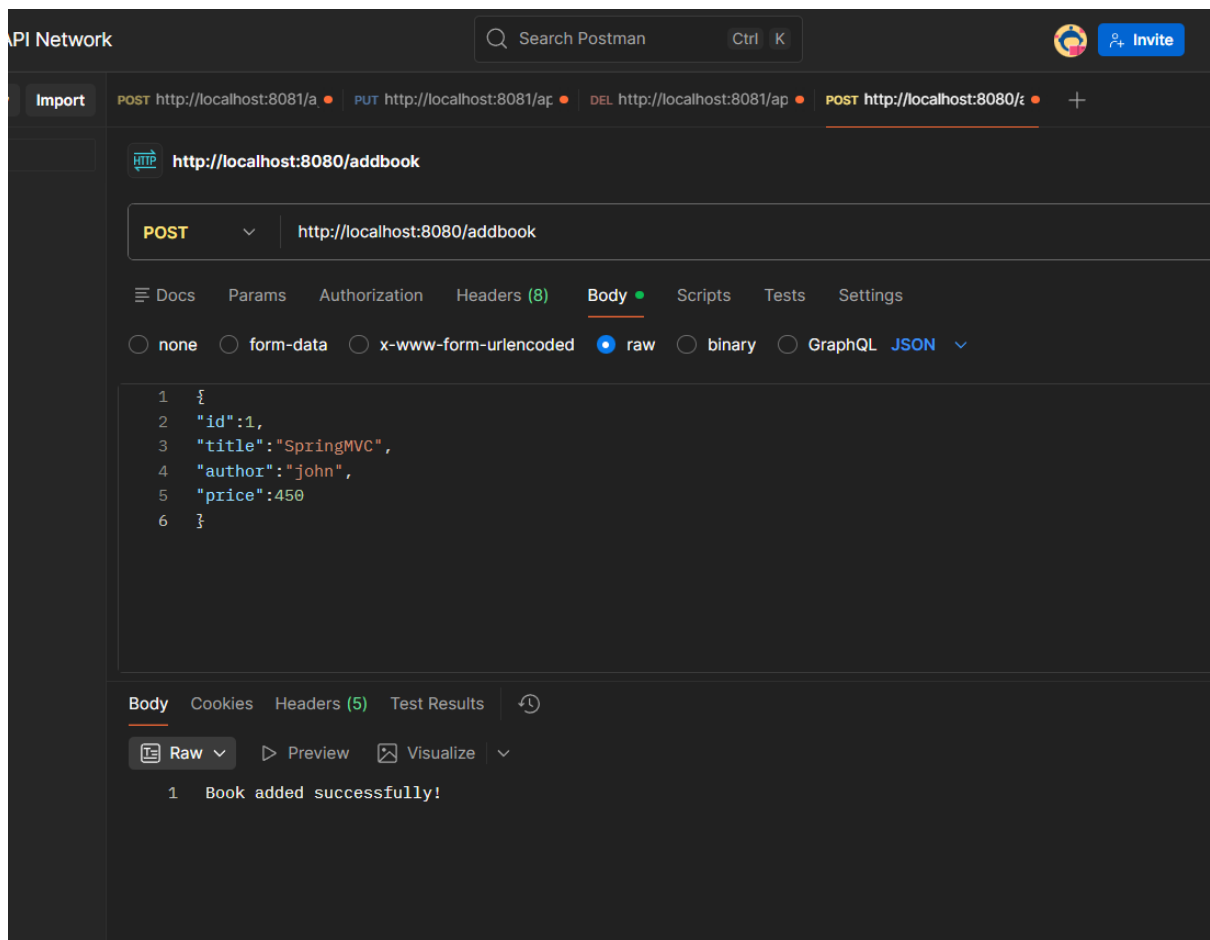
URL: <http://localhost:8080/addbook>

Method : POST

Body → JSON

```
{
  "id":1,
  "title":"Spring Boot",
  "author":"Rod Johnson",
  "price":450
}
```

Click **Send**



Method: GET

URL: <http://localhost:8080/viewbooks>

APINetwork

Search Postman

Ctrl + R

Invite

NewImport

POST http://localhost:8081/a

PUT http://localhost:8081/ap

DEL http://localhost:8081/ap

POST http://localhost:8080/i

GET http://localhost:8080/vi

HTTP

http://localhost:8080/viewbooks

GET

http://localhost:8080/viewbooks

Docs

Params

Authorization

Headers (6)

Body

Scripts

Tests

Settings

Query Params

	Key	Value	Description
	Key	Value	Description

Body

Cookies

Headers (5)

Test Results

200

{ } JSON

Preview

Visualize

```
1  {
2    {
3      "id": 1,
4      "title": "SpringMVC",
5      "author": "john",
6      "price": 450.0
7    },
8    {
9      "id": 1,
10     "title": "SpringMVC",
11     "author": "john",
12     "price": 450.0
13   }
14 }
```